

Offline Cryptography CTF Report

1. Student and Assignment

- Student name: **Nabiha Noor Khalique**
- Roll number: **23L-0967**
- Section: **7D**
- Assignment: **Offline Cryptography CTF**
- Case: **NORTHSTAR**

2. Investigation Process

The local case portal identified three independent cipher branches: monoalphabetic substitution, Vigenere, and Rail Fence. The archive also supplied a final hexadecimal ciphertext. The case data specified that the three hexadecimal fragments had to be ordered by their labelled positions and used as a byte-wise XOR pad.

The supplied evidence was kept unchanged in the `assets` and `data` directories. The programs were written as independent Python tools and run locally without network services.

3. Monoalphabetic Substitution

The ciphertext was read from `assets/mono\_ciphertext.txt`. The program calculated and printed letter frequencies, then applied a cipher-to-plaintext substitution map. The case supplied two confirmed cribs:

- `zwwazuae` -> `APPEARED`
- `NUZGOALQ` -> `FRAGMENT`

Additional mappings were checked against repeated occurrences and word patterns. The completed map was used by `mono\_solver.py` to calculate the recovered plaintext. The result included the mono stage flag and the fragment at position 1.

The mono stage produced:

- Flag: `CTF{MONO\_a2e3003267024e3d0df6dfe740ccb0ce}`
- Fragment 1: `9154793b7e4628342c99b15af89d7a14ea98e0eb7fd16675d5c3cc86067dc809b867f`

4. Vigenere Decryption

The case note specified the formula $K_i = (d_i + i) \bmod 26$, where the repeated roll-number digit sequence is `230967`, `i` starts at zero, and the key length is 10 letters. Nonletters are preserved and do not advance the key.

Using A=0 through Z=25, the derived key was:

```
`CECMKMIKIS`
```

`vigenere\_solver.py` used this calculated key to decrypt the complete ciphertext. It produced:

- Flag: `CTF{VIGENERE\_80CA10D7058C04A89925928BC2D60E25}`
- Fragment 2: `e39876754011d75fdc1d5a749bdaa8f90c39e7630051b7aa41c3e47bf294800fa8ff87b5`

5. Rail Fence Decryption

`rail\_solver.py` implemented standard down-and-up Rail Fence decryption and printed candidates for every depth from 2 through 8. The candidate at depth 4 was meaningful and stated that the correct depth was 4.

The depth-4 result produced:

- Flag: `CTF{RAIL\_486a5ae37d020091ecdea5b53f55c760}`
- Fragment 3: `0f50ba67a7e6592c498bec6d9642d6e2d5040dbe68a56a42a03a4d35db61e06dd0caa8`

6. Ordered Fragment Reconstruction and XOR

The fragments were concatenated in their labelled order 1, 2, and 3. The combined hexadecimal pad decoded to 107 bytes. The final ciphertext from the archive also decoded to 107 bytes, satisfying the required length check.

`otp\_solver.py` decoded both values and XORed corresponding bytes:

```
`plaintext_byte = ciphertext_byte XOR pad_byte`
```

The recovered message was:

```
`ACCESS GRANTED. Investigation complete for 23L-0967.  
FINAL_FLAG=CTF{FINAL_7d71401726938add299b6764988ca138}`
```

7. Theoretical Questions

AES block and key sizes

AES uses a 128-bit block size and standardized key sizes of 128, 192, and 256 bits. AES is preferred to DES because it provides much larger key sizes, a stronger modern design, and practical security against brute-force attacks. AES is also standardized and efficiently implemented in software and hardware.

DES effective key size

DES has an effective key size of 56 bits. Modern hardware can search that keyspace through brute force in a practical amount of time, so DES is no longer adequate for protecting modern systems.

Confusion and diffusion

Confusion obscures the relationship between the key and ciphertext, usually through substitutions and nonlinear operations. Diffusion spreads the influence of each plaintext bit across many ciphertext bits, usually through permutations and mixing. Together they make patterns and statistical attacks harder.

One-time-pad key length and reuse

An OTP key must be at least as long as the plaintext so every plaintext byte is masked by an independent key byte. It must be used only once because reuse creates relationships between ciphertexts.

Reusing an OTP key

If the same key encrypts two messages, an attacker can calculate $C1 \text{ XOR } C2$, because $C1 \text{ XOR } C2 = P1 \text{ XOR } P2$. This removes the key and can reveal plaintext patterns or allow recovery of both messages when parts of one plaintext are guessed.

8. Conclusion

The investigation combined frequency analysis, substitution consistency, a roll-derived Vigenere key, Rail Fence depth testing, and ordered byte-wise XOR. All recovered values were calculated by the submitted programs rather than printed as fixed plaintext answers.